Prompt for Windsurf:

You are taking over development from another AI agent on a project called **PulseCare Telehealth Scheduler**, a mid-complexity healthcare application built with:

- **Backend**: Java 21, Spring Boot 3 microservices, Maven, Spring Cloud (Eureka, Config, Gateway)
- **Frontend**: React (Staff Portal) + Vue.js (Patient Portal)
- **Databases**: PostgreSQL for transactional data, MongoDB for audit logs
- **Deployment**: Docker, Kubernetes, GCP (GCR + GKE), GitHub Actions CI/CD
- **Scaling**: Horizontal Pod Autoscaler (HPA)
- **Monitoring**: Prometheus + Grafana
- **HIPAA-friendly**: Role-based JWT auth, PII masking
- **Target**: PST/EST work hours alignment

Current state:

- Repo: https://github.com/sachinschitre/pulsecare-telehealth
- Directory structure, many backend services, some Docker and K8s manifests are created.
- .env file is populated with GCP params.
- GitHub Actions secrets set.
- GCP project created.
- Some scripts (`make init`, GCP deploy scripts) fail because of missing files/configs.
- Frontend code (React & Vue) is not yet implemented — only folder placeholders exist.
- Local testing via `docker-compose` fails because of service misconfigurations and missing dependencies.

Your task:

1. **Audit & Fix Missing Pieces**
    - Identify and create any missing files, configs, Dockerfiles, and Kubernetes manifests so the entire system builds and runs without error.
    - Ensure each microservice has:
        - `pom.xml` with correct dependencies.
        - `application.yml` / `application.properties`.
        - Dockerfile with multi-stage build (distroless or Alpine base).
        - Working Spring Boot entrypoint.
    - Ensure all Docker services build and start with `make up`.
2. **Frontend Implementation**
    - Build **React Staff Portal**:
        - Login (Admin/Provider) with JWT auth.
        - Provider availability management.
        - Appointment management dashboard.
        - Real-time telehealth session launch (Jitsi).
    - Build **Vue Patient Portal**:
        - Login/Register as Patient.
        - Search providers by specialty & availability.
        - Book/cancel appointment.
        - Join telehealth session.
    - Use **sleek, modern UI** (TailwindCSS or Material UI).

- Implement API integration with backend gateway.

3. **Local Development (Docker)**
    - Make `docker-compose.yml` fully functional for local dev with:
        - PostgreSQL + MongoDB init scripts.
        - All backend services.
        - Both frontends.
        - API Gateway.
        - Config & Discovery services.
    - Ensure `make init` installs all dependencies, `make up` runs full stack, and `./scripts/local_seed.sh` seeds demo data.
4. **Testing**
    - Add JUnit 5 + Testcontainers for backend integration tests.
    - Add Playwright or Cypress E2E tests for frontends.
    - Ensure `make test` runs all tests locally and in CI.
5. **Deployment to GCP**
    - Fix and complete scripts for:
        - Building & pushing images to **Google Container Registry**.
        - Deploying all services to **Google Kubernetes Engine**.
        - Applying Kubernetes manifests (`kubectl apply -f ...`).
        - Setting up Ingress + SSL (self-signed if needed for demo).
    - After deploy, app should be accessible via public URL and HPA scaling demonstrable.

6. **Metrics & Monitoring**
    - Deploy Prometheus + Grafana to GKE.
    - Provide a sample Grafana dashboard JSON for monitoring CPU, memory, request count, and HPA pod count.
7. **Documentation**
    - Update `README.md` with:
        - Project overview.
        - Local dev instructions.
        - Testing instructions.
        - GCP deploy guide.
        - Demo script for interview presentation.
8. **Final Output**
    - Fully working app locally and on GCP.
    - Frontend + backend integrated.
    - CI/CD passing.
    - Demo-ready in 1–2 hours of your work.

Efficiency Metrics Requirement (for Interview Presentation):

- Include in the final README a **"Time & Efficiency Gains"** section that:
    - States that **manual development of this stack with one junior dev would take ~16 hours (2 full days)**.
    - States that using AI-assisted dev + strong human direction reduced it to **~2 hours**, an **88% time savings**.

    - Break down the time saved by stage (architecture setup, backend build, frontend build, deployment).
    - Present the result as:
        - **Original Estimate:** 16 hours
        - **Actual Delivery:** 2 hours
        - **Productivity Multiplier:** 8× faster delivery
        - **Efficiency Gain:** 88% reduction in dev time

Important Constraints:

- You should optimize for speed: reuse existing code where possible, don't over-engineer.
- Use secure coding & HIPAA-friendly practices.
- Output all missing or modified files directly in this workspace.
- Run `make up` and `make test` locally to verify before final GCP deploy.